Práctica: API REST — Juego de la Mosca

1. Descripción

Desarrolla una **API REST utilizando PHP** para gestionar un juego denominado **La Mosca**.

La aplicación estará compuesta por un backend que gestionará:

- Usuarios y sus permisos.
- Partidas/tableros de cada usuario.
- El juego de "La Mosca".
- Los intentos disponibles.
- La resolución de las partidas.

No es necesario desarrollar una interfaz gráfica. La API deberá poder probarse mediante herramientas como **Postman**.

---

2. El juego

El juego consiste en descubrir en qué casilla del tablero se encuentra escondida una mosca.

Por defecto, un tablero tendrá:

- **10 casillas**.
- **5 intentos**.
- Una mosca situada aleatoriamente en una de las casillas.

El usuario no conocerá inicialmente la posición de la mosca.

Por ejemplo, un tablero podría ser:

```
[ ][ ][ ][ ][ ][ ][ ][ ][ ][ ]
```

La mosca podría estar escondida en:

```
[ ][ ][ ][🐝][ ][ ][ ][ ][ ][ ]
```

pero el usuario no podrá conocer esta posición hasta que la descubra o se rinda.

---

3. Golpear una casilla

El jugador podrá realizar un intento indicando una casilla del tablero.

Por ejemplo:

```
{
    "casilla": 4
}
```

El backend deberá comprobar la posición de la mosca y responder con uno de los siguientes resultados:

🪰 Cazada

El jugador ha encontrado la mosca.

```
cazada
```

La partida se considera **finalizada**.

🟠 Casi le das

La casilla elegida está cerca de la posición de la mosca.

```
casi le das
```

En este caso, **la mosca revolotea**, es decir, cambia de posición.

El cambio de posición deberá realizarlo el backend.

El alumno deberá establecer y documentar qué considera una casilla "cercana" a la mosca.

⚪ Ni se entera

El jugador ha golpeado una casilla que está suficientemente alejada de la mosca.

```
ni se entera
```

La mosca permanece en su posición.

Cada golpe realizado consume **un intento**.

Cuando el jugador se queda sin intentos, la partida termina.

---

4. Usuarios

La API deberá disponer de un sistema de usuarios.

Existirán dos tipos de usuario:

```
ADMIN
JUGADOR
```

Jugador

Un jugador podrá:

- Registrarse.
- Iniciar sesión.
- Gestionar sus tableros.
- Jugar sobre su tablero activo.
- Consultar sus intentos restantes.
- Rendirse de una partida.

Administrador

El administrador tendrá permisos para:

- Crear usuarios.
- Consultar usuarios.
- Modificar usuarios.
- Eliminar usuarios.
- Cambiar el tipo de usuario entre `ADMIN` y `JUGADOR`.
- Los usuarios podrán registrar y realizar login.
- También se podrá recuperar la contraseña que se enviará por email.

Los usuarios también podrán registrarse mediante la API, pero deberán comenzar con el rol de **JUGADOR**.

Un jugador **no podrá registrarse directamente como administrador**.

---

5. Tableros de los usuarios

Cada usuario podrá tener varios tableros almacenados.

Sin embargo:

> **Un usuario solamente podrá tener un tablero activo simultáneamente.**

Un tablero tendrá, como mínimo, información equivalente a:

- Usuario al que pertenece.

- Tamaño.
- Número de intentos iniciales.
- Intentos restantes.
- Posición de la mosca.
- Estado del tablero.
- Fecha de creación.

Los estados del tablero podrán ser, por ejemplo:

```
ACTIVO
GANADO
PERDIDO
RENDIDO
```

Se podrá modificar esta propuesta si considera que existe una solución mejor, pero deberá justificarla.

---

6. Crear un tablero

El jugador podrá crear un nuevo tablero mediante:

```
POST /crear
```

El cuerpo de la petición deberá permitir indicar:

```
{
    "tamano": 15,
    "intentos": 8
}
```

Los valores serán personalizados.

Si el jugador no especifica alguno de ellos, se utilizarán los valores por defecto:

```
tamaño:   10
intentos: 5
```

Restricción

Si el usuario ya tiene un tablero activo:

```
POST /crear
```

deberá devolver un error y **no deberá crear otro tablero activo**.

Se deberá decidir qué código HTTP se considera apropiado y justificarlo.

---

7. Jugar

El jugador realizará un golpe mediante una petición:

```
POST /...
```

El cuerpo contendrá la casilla seleccionada:

```
{
    "casilla": 7
}
```

El backend deberá:

1. Comprobar que el usuario está autenticado.
2. Comprobar que tiene un tablero activo.
3. Validar que la casilla existe.
4. Comprobar la posición de la mosca.
5. Restar un intento.
6. Determinar el resultado:
    - `cazada`
    - `casi le das`
    - `ni se entera`
7. Si corresponde, mover la mosca.
8. Comprobar si se ha terminado la partida.
9. Devolver una respuesta adecuada al cliente.

Importante

El cliente **nunca debe recibir la posición de la mosca mientras la partida esté activa**.

Por ejemplo, una respuesta podría ser:

```
{
    "resultado": "casi le das",
    "intentos_restantes": 3
}
```

El formato definitivo de las respuestas queda a elección del alumno.

---

8. Consultar intentos

El jugador podrá consultar los intentos que le quedan mediante:

```
GET /intentos
```

La respuesta deberá indicar los intentos restantes del tablero activo.

Por ejemplo:

```
{
    "intentos_restantes": 3
}
```

Si el usuario **no tiene ningún tablero activo**, la API deberá informar adecuadamente de esta situación.

---

9. Rendirse

El jugador podrá abandonar la partida mediante:

```
PUT /rendirse
```

Si existe un tablero activo:

1. La partida finalizará.
2. El tablero pasará a un estado que indique que el jugador se ha rendido.
3. El backend devolverá al cliente la información necesaria para que pueda **ver dónde estaba escondida la mosca**.

Por ejemplo:

```
{
    "estado": "RENDIDO",
    "mosca": 8
}
```

Se podrá elegir cómo representar el tablero.

Si no existe ningún tablero activo, deberá devolverse el error correspondiente.

---

10. CRUD de usuarios

La API deberá proporcionar las operaciones necesarias para realizar el CRUD de usuarios.

Como mínimo deberá ser posible:

Crear

Registrar un usuario.

```
POST /usuarios
```

Consultar

Obtener uno o varios usuarios.

```
GET /usuarios
GET /usuarios/{id}
```

Modificar

Modificar los datos permitidos de un usuario.

```
PUT /usuarios/{id}
```

Eliminar

Eliminar un usuario.

```
DELETE /usuarios/{id}
```

Estas operaciones estarán **restringidas al administrador**, salvo aquellas relacionadas con el registro o gestión de los propios datos del usuario que decidáis permitir.

---

11. Gestión de permisos

El administrador deberá poder cambiar el rol de un usuario.

Por ejemplo:

```
{
    "rol": "ADMIN"
}
```

Un usuario normal **no podrá modificar su propio rol**.

Tampoco podrá modificar el rol de otro usuario.

La API deberá controlar estas restricciones mediante autenticación y autorización.

---

12. Autenticación

La API deberá implementar un mecanismo de autenticación.

Las rutas que requieran autenticación deberán rechazar las peticiones realizadas por usuarios no autenticados.

Además, las rutas de administración deberán comprobar que el usuario autenticado tiene permisos de administrador.

---

13. Reglas importantes

La aplicación deberá cumplir, como mínimo, las siguientes reglas:

1. Un usuario no autenticado no puede jugar.
2. Un usuario no autenticado no puede consultar sus tableros.
3. Un jugador no puede gestionar los usuarios de otros jugadores.
4. Un jugador no puede cambiar su rol.
5. Un jugador no puede crear dos tableros activos.
6. Un jugador solo puede jugar en su propio tablero.
7. Un jugador no puede conocer la posición de la mosca mientras la partida esté activa.
8. Un golpe consume un intento.
9. Cuando no quedan intentos, la partida termina.
10. Una partida terminada no puede seguir recibiendo golpes.

11. Al rendirse, el jugador recibe la posición de la mosca.
12. Un jugador sin tablero activo debe recibir un error al intentar jugar.
13. Un jugador sin tablero activo debe recibir un error al consultar los intentos.
14. Un jugador sin tablero activo debe recibir un error al intentar rendirse.
15. El tamaño del tablero y el número de intentos deben validarse.
16. Las casillas fuera del tablero deben rechazarse.

---

14. Base de datos

Se deberá diseñar la base de datos necesaria para implementar el sistema.

Como mínimo deberán existir entidades equivalentes a:

```
USUARIOS
TABLEROS
```

El diseño deberá incluir las relaciones necesarias entre ambas.

Se deberá entregar:

- BD exportada.
- Datos iniciales para poder probar la aplicación.

Deberá existir, como mínimo, un usuario administrador de prueba.

---

15. Respuestas HTTP

La API deberá utilizar correctamente los códigos de estado HTTP.

Por ejemplo:

```
200 OK
201 Created
400 Bad Request
401 Unauthorized
403 Forbidden
404 Not Found
409 Conflict
422 Unprocessable Entity
```

No es necesario utilizar exactamente estos códigos en todos los casos, pero se deberán **elegir códigos coherentes con cada situación y documentar las decisiones importantes**.

Las respuestas deberán utilizar JSON.

16. Documentación de la API

El proyecto deberá incluir documentación de todos los endpoints.

Para cada endpoint se indicará:

- Método HTTP.
- URL.
- Si requiere autenticación.
- Rol necesario.
- Parámetros.
- Body.
- Respuesta correcta.
- Posibles errores.
- Código HTTP utilizado.

Por ejemplo:

```
POST /crear

Autenticación: Sí
Rol: JUGADOR

Body:

{
    "tamano": 10,
    "intentos": 5
}

Respuesta:

201 Created

{
    "mensaje": "Tablero creado correctamente"
}
```

17. Pruebas

Deberán realizarse pruebas de la API utilizando Postman

Como mínimo se deberán probar:

- Registro de usuario.
- Login.

- Login incorrecto.
- Acceso sin autenticación.
- CRUD de usuarios como administrador.
- Intento de CRUD como jugador.
- Cambio de permisos.
- Creación de tablero.
- Intento de crear un segundo tablero activo.
- Golpe válido.
- Golpe fuera del tablero.
- Golpe cuando no hay tablero activo.
- Consulta de intentos.
- Rendición.
- Rendición sin tablero activo.
- Finalización por falta de intentos.
- Victoria al encontrar la mosca.
- Intento de seguir jugando después de finalizar la partida.

---

18. Entrega

El proyecto deberá incluir:

```
/proyecto
    ├── código fuente
    ├── datos de prueba
    ├── README.md
    ├── documentación API
    └── colección de Postman
```

El `README.md` deberá explicar:

- Cómo instalar el proyecto.
- Cómo configurar la base de datos.
- Cómo arrancar la API.
- Usuarios de prueba.
- Cómo autenticarse.
- Cómo probar los endpoints.